Sim-to-Real Quadrupedal Visual Navigation via 3DGS and RL Jitter

Minseok Lee[1], Seongyu Han[1], Heejae Moon[1], and Sung Soo Hwang[1*]

[1]School of Artificial Intelligence, Computer and Electrical Engineering,
Handong Global University, Pohang 37554, Republic of Korea
({glen, tjsrb439, 22000249}@handong.ac.kr, sshwang@handong.edu) * Corresponding author

Abstract: Deploying visual navigation on physical quadrupedal robots is challenging due to the visual sim-to-real gap and locomotion-induced camera jitter. This jitter degrades feature tracking and depth consistency. This paper presents a pipeline that bridges these perceptual and dynamic gaps to enable virtual parameter optimization. In the simulation stage, we reconstruct a 3D Gaussian Splatting (3DGS) digital twin of a campus corridor and use a reinforcement learning (RL) locomotion policy to generate camera bobbing noise. Under these simulated visual and dynamic disturbances, the parameters of the RTAB-Map visual SLAM and Nav2 stacks are optimized. For deployment, the RL policy is bypassed, and velocity commands are routed directly to the physical robot's built-in Sport API via a ROS 2 bridge. Real-world experiments demonstrate that parameters optimized under simulated jitter transfer directly to the physical platform, achieving visual mapping and obstacle avoidance without manual recalibration.

Keywords: Quadrupedal Locomotion, Visual SLAM, Sim-to-Real Pipeline, 3D Gaussian Splatting, Ego-motion Jitter, Virtual Sandbox.

1. INTRODUCTION

Autonomous quadrupedal robots offer superior mobility over wheeled systems on challenging terrains. Achieving indoor autonomy requires integrating legged locomotion with simultaneous localization and mapping (SLAM). While indoor navigation has conventionally relied on LiDAR sensors, they are heavy and power-intensive. Visual navigation using a single RGB-D camera is a lightweight, cost-effective alternative.

However, legged visual navigation introduces two primary sim-to-real challenges. First is the *perceptual gap*: standard simulators lack photorealistic textures and depth fidelity, causing visual SLAM to fail in the real world due to feature matching errors. Second is the *dynamic gap*: legged locomotion generates camera bobbing that causes motion blur and false obstacle detection (phantom obstacles). Furthermore, tuning parameters directly on physical hardware is limited by battery capacity and collision risks.

To address these challenges, we present a pipeline that decouples simulation-based parameter verification from physical deployment. Using a single front-facing RGB-D camera (Intel RealSense D435i) and offline 3DGS reconstruction from DSLR images, we align virtual and physical camera properties to reduce sensor discrepancy. In simulation, we reconstruct a digital twin of an indoor corridor. Since the robot's low-level controller is proprietary, we train an RL locomotion policy in Isaac Lab to emulate physical gait oscillations, generating representative camera noise. In this virtual environment, we optimize the RTAB-Map and Nav2 parameters. During physical deployment, the computationally intensive RL policy is bypassed, and velocity commands are routed directly to the manufacturer's Sport API via a ROS 2 bridge, conserving edge computing resources for SLAM and path planning.

The contributions of this work are as follows:

1. We integrate DSLR-based 3DGS and *fVDB* reconstruction into a physics simulator, aligning virtual camera parameters with the physical sensor to establish a photo-realistic validation environment.
2. We train an RL locomotion policy to emulate walking dynamics, generating representative camera jitter in simulation to bridge the dynamic gap.
3. We demonstrate that navigation and mapping parameters optimized under simulated disturbances transfer directly to a physical Unitree Go2 without recalibration.

The remainder of this paper is organized as follows: Section II reviews related work. Section III describes the system architecture, locomotion learning, and environment construction. Section IV details the navigation configuration, simulated validation, and real-world deployment results. Finally, Section V concludes this paper.

2. RELATED WORK

Reinforcement learning (RL) has advanced legged control [1], enabling robust policies without accurate dynamics models. Highly parallelized simulators like NVIDIA Isaac Sim and Isaac Lab facilitate deep RL training for sim-to-real transfer on platforms like ANYmal and Unitree Go1/Go2. However, most legged RL research focuses on gait stability, leaving visual navigation integration and sensor noise characteristics unaddressed.

Mobile robot navigation has traditionally relied on LiDAR sensors, which are heavy and power-intensive for payload-constrained quadruped robots. Visual SLAM (VSLAM) using RGB-D cameras offers a lightweight alternative. Real-Time Appearance-Based Mapping (RTAB-Map) [2] combines appearance-based loop closure detection with pose-graph optimization to minimize drift. For path planning and obstacle avoidance, the ROS 2 Navigation (Nav2) framework [3] is widely used.

Although frameworks like VR-Robo [5] use 3DGS [4]

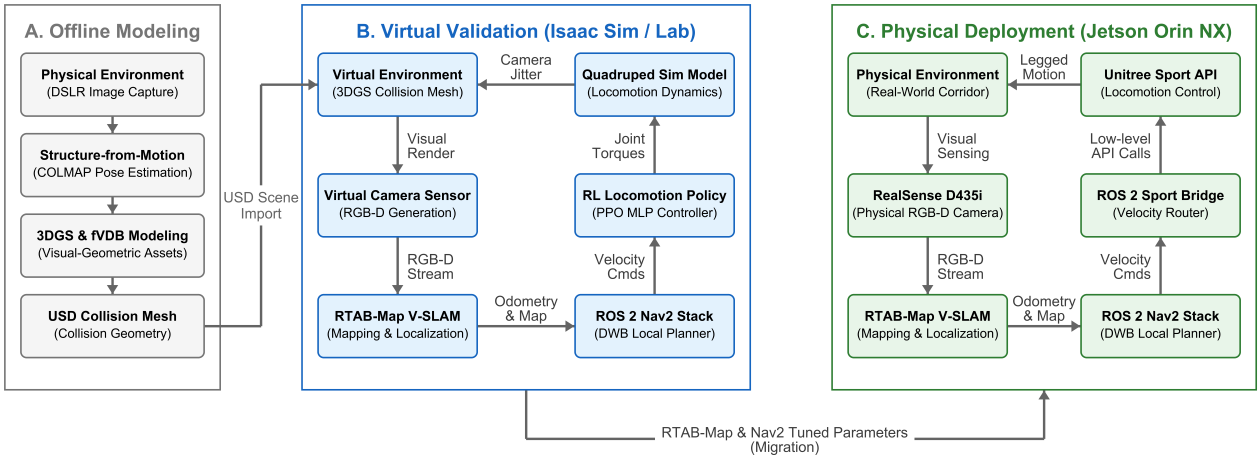

Fig. 1. Overall architecture of the proposed Sim-to-Real quadrupedal navigation framework. (a) In the virtual validation stage, we reconstruct a 3DGS environment and train an RL locomotion policy to emulate camera bobbing noise. (b) In the physical deployment stage, the RL policy is bypassed, and commands are sent to the Unitree Sport API via a ROS 2 bridge to reduce computational load on the Jetson Orin NX.

to build digital twins, they often rely on heterogeneous sensors (e.g., using an iPad for modeling and an action camera for deployment). This introduces device-level discrepancies in lens distortion and field-of-view (FOV). In contrast, our pipeline uses a DSLR camera for 3DGS reconstruction and configures the virtual camera to match the physical D435i sensor, minimizing visual discrepancies.

3. SYSTEM AND LOCOMOTION CONTROL

3.1 System Architecture

Our framework integrates RTAB-Map, Nav2, and a dual-stage locomotion control architecture. In simulation, a DRL policy—trained via PPO in Isaac Lab (v2.3.2) on Isaac Sim (v5.1.0)—generates camera bobbing noise. High-level velocity commands ($\mathbf{v_{cmd}} = [\mathbf{v_x}, \mathbf{v_y}, \omega_\mathbf{z}]^\mathbf{T}$) from the Nav2 local planner are mapped directly to the policy's action space. During real-world deployment, the DRL policy is bypassed, and velocity commands are routed directly to the Unitree Sport API via a ROS 2 bridge. This architecture reduces the computational load on the Jetson Orin NX. The system architecture is shown in Fig. 1.

3.2 Locomotion Learning in Isaac Sim

To simulate camera bobbing noise, we train an RL policy in NVIDIA Isaac Lab (v2.3.2) on the Isaac Sim (v5.1.0) physics engine. The policy functions to generate walking-induced oscillations. We construct a training environment with mixed terrains (uneven ground, ramps, step obstacles) using the Procedural Terrain Generator and scale terrain difficulty via curriculum learning. To enhance policy robustness, we apply the following domain randomizations:

- Torso mass randomization: $\Delta m \in [-1.0, +3.0]$ kg.
- Ground friction coefficient: $\mu \in [0.6, 0.8]$.
- External disturbances: impulse force perturbations applied to the robot's center of mass (CoM).

The physics step is 200 Hz, and the policy executes at 50 Hz. Training uses the PPO implementation in the PyTorch-based *skrl* library. Both actor and critic networks are MLPs with hidden layers of $[512, 256, 128]$ and ELU activation functions. The learning rate is 1.0×10^{-3}, regulated by a KL-divergence scheduler. The discount factor is $\gamma = 0.99$, and the GAE parameter is $\lambda = 0.95$.

The reward function tracks velocity references while maintaining legged stability. The total reward R is:

$$R = w_{lin}R_{lin} + w_{ang}R_{ang} + w_{air}R_{air} - \sum_i w_{p,i}P_i \quad (1)$$

where:

- $R_{lin} = \exp(-\|\mathbf{v}^*_{\mathbf{xy}} - \mathbf{v}_{\mathbf{xy}}\|^2_2/\sigma^2_{\mathbf{lin}})$ is the linear velocity tracking reward ($\mathbf{v}^*_{\mathbf{xy}}$ and $\mathbf{v}_{\mathbf{xy}}$ are target and actual velocities, $w_{lin} = 1.5, \sigma_{lin} = 0.25$).
- $R_{ang} = \exp(-(\omega^*_z - \omega_z)^2/\sigma^2_{ang})$ is the angular velocity tracking reward (ω^*_z and ω_z are target and actual yaw rates, $w_{ang} = 0.75, \sigma_{ang} = 0.25$).
- R_{air} is the foot clearance air-time reward ($w_{air} = 0.25$).
- P_i represents penalty terms for base vertical velocity, roll/pitch oscillations, joint limits, excessive torque, joint acceleration, and unintended contact ($w_{p,i}$ are penalty weights).

3.3 3DGS Environment Construction & Agent Deployment

We capture corridor images using a DSLR camera to avoid compression artifacts. These images serve as input to COLMAP for Structure-from-Motion (SfM) to resolve camera poses. Then, a 3DGS [4] model is optimized and exported as a USDZ asset for visual rendering, alongside a PLY mesh for coarse geometry. For physical collision modeling, we integrate the scene with NVIDIA's sparse volumetric framework, *fVDB* (Fig. 2).

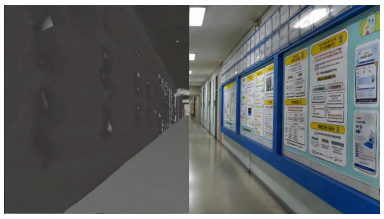

Fig. 2. Digital twin representation of the campus corridor. Left: Sparse volumetric collision mesh generated using fVDB. Right: 3DGS render.

We import these assets into Isaac Sim, where the collision boundaries are computed using the *fVDB* representation. A ROS 2 bridge publishes virtual RGB-D sensor streams, visualizes the robot state in RViz, and routes velocity commands. The virtual camera's field-of-view and resolution are configured to match the physical D435i camera to reduce the visual gap. The pre-trained locomotion policy is then deployed on the Go2 model to maintain stable gaits within the simulated corridor.

4. EXPERIMENTAL RESULTS

4.1 Experimental Setup and Evaluation Methodology

The navigation system consists of an Intel RealSense D435i RGB-D sensor, the RTAB-Map framework [2], and the ROS 2 Nav2 stack. The sensor is mounted to the front torso of the Go2. In simulation, the virtual camera replicates the physical sensor's color and depth profiles. This setup ensures depth-texture consistency. Unlike monocular frameworks (e.g., VR-Robo [5]) that suffer from depth ambiguity in textureless regions, the active depth sensor provides geometric measurements for local costmaps.

RTAB-Map extracts visual keypoints (e.g., GFTT/ORB) from the RGB stream for odometry and loop-closure detection. The depth stream is projected to construct a 3D octomap and a 2D occupancy grid. For low-latency obstacle avoidance, raw depth frames are converted into a pseudo-2D laser scan via the *depthimage_to_laserscan* package and published to the */scan* topic. The Nav2 global planner generates paths on the occupancy grid, while the DWB local planner uses the local costmap to compute steering commands $\mathbf{v_{cmd}}$.

To resolve the coordinate transform (TF) tree discrepancy between the simulator and ROS 2, we implement a custom bridge. Nav2 expects *map* $\rightarrow$ *odom* $\rightarrow$ *base*, whereas Isaac Sim tracks the robot relative to the simulator's *world* frame. Our bridge maps the tree as *map* $\rightarrow$ *odom* $\rightarrow$ *world* $\rightarrow$ *base*, where RTAB-Map publishes the *map* $\rightarrow$ *odom* transform, and Isaac Sim links odometry to the world frame. This enables standard ROS 2 navigation in simulation without modification. The system is evaluated by analyzing mapping consistency and obstacle avoidance success in both the simulated 3DGS corridor and the physical environment under walking-induced camera oscillations, measuring the transferability of the optimized parameters.

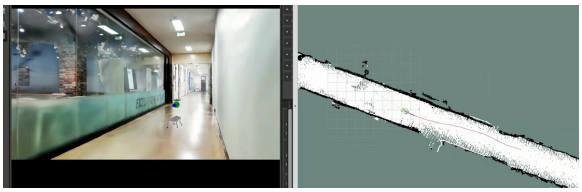

Fig. 3. Autonomous navigation execution inside the reconstructed virtual environment, showing the simulated quadruped robot in Isaac Sim (left) and path planning/costmap synchronization in RViz (right).

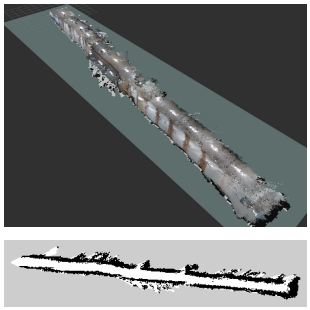

Fig. 4. Visual mapping results in simulation. Top: Reconstructed 3D point cloud map from RTAB-Map. Bottom: Projected 2D occupancy grid map.

4.2 Results and Evaluation

We validated the navigation stack in a simulated 3DGS corridor (80 m long, 2.5 m wide) before physical deployment.

In simulation, the locomotion policy receives proprioceptive and exteroceptive (height-scan) inputs. Raycasting queries the collision mesh to provide local elevation profiles for foot placement. The velocity commands are supplied by the DWB planner (Fig. 3). To evaluate obstacle avoidance, virtual box obstacles were placed along the corridor. The local planner recalculated trajectories, and the policy adjusted its walking pattern to avoid obstacles (Fig. 3). The resulting point cloud and occupancy grid maps are shown in Fig. 4.

We deployed the pipeline on a physical Unitree Go2 in a campus corridor (Fig. 5). The onboard hardware consists of an NVIDIA Jetson Orin NX (20W mode, 16GB RAM) running Ubuntu 20.04 and ROS 2 Foxy. To reduce computational load, the RL policy was bypassed on the onboard Jetson Orin NX. Instead, velocity commands were routed directly to the Unitree Sport API using a ROS 2 bridge, reserving computing resources for SLAM and path planning. The D435i sensor was connected via USB 3.0.

Although the simulation used ROS 2 Jazzy (Ubuntu 24.04) and the physical robot used ROS 2 Foxy (Ubuntu 20.04), the same software architecture was maintained. This allowed parameters optimized under simulated camera noise to transfer directly to the physical platform without manual recalibration. RTAB-Map successfully detected loop closures, compensating for walking oscil-

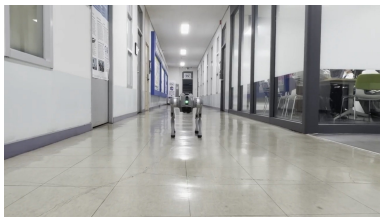

Fig. 5. Real-world experimental setup. The Unitree Go2 robot is equipped with an Intel RealSense D435i and Jetson Orin NX, running the entire navigation stack on-board.

lations. The robot maintained a target velocity of 0.4 m/s. These results indicate that parameters optimized in simulation transfer directly to the physical robot.

4.3 Limitations

While the proposed pipeline successfully bridges the sim-to-real gap, several limitations remain. First, bypassing the RL controller on-board the physical robot limits its adaptive locomotion capabilities, relying instead on the manufacturer's proprietary Sport API. Second, our navigation stack was evaluated primarily in static environments; dynamic obstacle avoidance and moving crowd navigation are left for future work. Finally, high-resolution rendering and SLAM are computationally constrained, necessitating edge hardware optimization.

5. CONCLUSION

This paper presented a dual-stage visual navigation framework for quadrupedal robots. By combining a 3D Gaussian Splatting digital twin with an RL locomotion policy that emulates walking dynamics, we established a simulation environment to optimize visual SLAM and path planning. For deployment, we bypassed the RL policy and routed velocity commands directly to the robot's Sport API via a ROS 2 bridge to reduce the onboard computational load. Real-world experiments on a Unitree Go2 demonstrated that parameters optimized under simulated camera jitter transfer directly to the physical platform, achieving stable visual tracking and obstacle avoidance without recalibration.

Future work will focus on incorporating dynamic obstacle avoidance strategies and optimizing the processing pipeline to support higher-resolution visual feedback on the edge computer.

ACKNOWLEDGEMENT

This research was supported by the National Research Foundation of Korea (NRF) grant funded by the Korean government (MSIT) (No. RS-2025-24683458).

USE OF AI TOOLS

During the preparation of this work, the authors used Gemini (developed by Google) for grammar editing and style improvement of the manuscript. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the final text of the paper.

REFERENCES

[1] J. Hwangbo, J. Lee, A. Dosovitskiy, D. Bellicoso, V. Tsounis, V. Koltun, and M. Hutter, "Learning agile and dynamic motor skills for legged robots," *Science Robotics*, vol. 4, no. 26, p. eaau5872, 2019.

[2] M. Labbé and F. Pomerleau, "Online global loop closure detection for large-scale multi-session graph-based SLAM," *Autonomous Robots*, vol. 34, no. 3, pp. 183–200, 2013.

[3] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," *Science Robotics*, vol. 7, no. 66, p. eabm6074, 2022.

[4] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3D Gaussian splatting for real-time radiance field rendering," *ACM Transactions on Graphics*, vol. 42, no. 4, pp. 1–14, 2023.

[5] S. Zhu, L. Mou, D. Li, B. Ye, R. Huang, and H. Zhao, "VR-Robo: A real-to-sim-to-real framework for visual robot navigation and locomotion," *IEEE Robotics and Automation Letters*, vol. 10, no. 8, pp. 7875–7882, August 2025.